FOOTPRINT ESG: NEXT-GENERATION ENTERPRISE SUSTAINABILITY & ESG PERFORMANCE PLATFORM

A PROJECT REPORT

Submitted by

SHYAMKUMAR R (Registration No: 310624205258)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF INFORMATION TECHNOLOGY



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Bharathi Salai, Ramapuram, Chennai – 600089

BONAFIDE CERTIFICATE

Certified that this mini project report titled "FOOTPRINT ESG: NEXT-GENERATION ENTERPRISE SUSTAINABILITY & ESG PERFORMANCE PLATFORM" is the Bonafide work of "SHYAMKUMAR R (310624205258)", who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT	**SUPERVISOR**
Department of Information Technology, Easwari Engineering College, Ramapuram, Chennai – 600089.	Department of Information Technology, Easwari Engineering College, Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Bharathi Salai, Ramapuram, Chennai – 600089

CERTIFICATE OF VIVA-VOCE EXAMINATION

This is to certify that the Viva-Voce Examination of the Project Work entitled "FOOTPRINT ESG: NEXT-GENERATION ENTERPRISE SUSTAINABILITY & ESG PERFORMANCE PLATFORM" submitted by Mr. SHYAMKUMAR R (Reg. No: 310624205258) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in INFORMATION TECHNOLOGY was conducted on _____ (Date) in the Department of Information Technology, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner		**External Examiner**	
Name	:	Name	:
Designation	:	Designation	:
Address	:	Address	:

Date:

ABSTRACT

The rapid implementation of international environmental regulations (EU CSRD, US SEC climate disclosures, and BRSR) demands a modern, automated enterprise sustainability management infrastructure capable of addressing severe bottlenecks in carbon accounting, data governance, and multi-facility compliance. Traditional industrial reporting relies on manual spreadsheets, ad-hoc utility invoices, and disconnected supervisory logs, resulting in unverified emission figures, reporting latencies, and severe compliance risk.

FootprintESG is a cloud-native, high-concurrency Full-Stack web platform engineered to automate multi-facility environmental accounting and ESG performance tracking. Built upon the MERN (MongoDB, Express, React, Node.js) architecture reinforced with Vite and Socket.io, the platform introduces:

1. Scope 1 & Scope 2 Carbon Accounting: Automated calculation of direct fuels (gas, diesel) and purchased grid electricity in metric tons CO2e compliant with the GHG Protocol and ISO 14064-3.

2. Four-Eyes Governance & Audit Trail: Enforces strict organizational segregation of duties separating operational data coordinators from sustainability managers and accredited auditors.

3. Real-Time Utility Substation Streaming: Socket-driven live monitoring of high-voltage grid load (kW), rooftop solar generation, boiler natural gas, and water intake using continuous adjacent drift.

4. SBTi Trajectory & Multi-Facility Benchmarking: Tracks corporate 2030 Science-Based Target milestones alongside carbon intensity league tables (tCO2e/ton) and landfill waste diversion.

The platform delivers a responsive, enterprise light interface with sub-100ms API response times and instant client synchronization, ensuring an efficient digital backbone for sustainable industrial decarbonization.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of Easwari Engineering College for providing us with the opportunity and state-of-the-art facilities to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Department of Information Technology, for continuous encouragement, valuable suggestions, and steadfast support throughout our academic journey.

We are extremely thankful to our Project Guide, [Guide Name], Department of Information Technology, for valuable guidance, constructive criticism, and motivation throughout the conception and completion of this project.

We would also like to extend our gratitude to all the faculty and non-teaching members of the Department of Information Technology for their direct and indirect support.

Finally, we thank our parents and friends for their constant encouragement, moral support, and cooperation during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	1
	1.4 SCOPE OF THE PROJECT	1
	1.5 MOTIVATION	2
2	**RELATED WORK**	**3**
	2.1 LITERATURE REVIEW	3
	2.2 EXISTING SYSTEM	3
	2.3 PROPOSED SYSTEM	3
3	**SYSTEM ANALYSIS**	**4**
	3.1 SYSTEM REQUIREMENTS	4
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	4
4	**SYSTEM DESIGN**	**6**
	4.1 SYSTEM ARCHITECTURE	6
	4.2 MODULE DESCRIPTION	6
	4.3 CLASS DIAGRAM	7
	4.4 SEQUENCE DIAGRAM	8
5	**IMPLEMENTATION**	**9**
	5.1 IMPLEMENTATION OVERVIEW	9
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	9
6	**CONCLUSION & FUTURE WORK**	**10**
	6.1 CONCLUSION	10
	6.2 FUTURE ENHANCEMENTS	10
	APPENDIX - SOURCE CODE	**11**
	SCREENSHOTS OF THE PROJECT	**24**
	REFERENCES	**26**

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Corporate sustainability and environmental decarbonization have emerged as premier governance imperatives for global industrial manufacturing. Conglomerates face operational hurdles tracking Scope 1 direct fuel combustion and Scope 2 purchased electricity across multi-facility enterprises. The FootprintESG platform is a full-stack system unifying corporate carbon accounting, Four-Eyes audit workflows, live substation telemetry, and international compliance disclosures into a single digital hub.

1.2 PROBLEM DEFINITION

1. Fragmented Spreadsheets & Silos: Plant coordinators manually compile disparate monthly utility bills, boiler logs, and waste slips in spreadsheets, creating severe discrepancies.

2. Absence of Four-Eyes Governance: Without enforced role-based state machines, operational staff or administrators can modify reported environmental numbers, risking compliance penalties.

3. Stale Decoupled Telemetry: Substation power meters, natural gas boilers, and water withdrawal operate without real-time observability, blinding plant leadership to continuous resource waste.

1.3 OBJECTIVE

- • To engineer a high-throughput web application connecting plant coordinators, sustainability directors, and third-party auditors.
- • To implement an automated GHG accounting engine compliant with the GHG Protocol Corporate Standard, ISO 14064-3, and SBTi.
- • To integrate Four-Eyes approval mechanics with immutable audit logs preventing unverified records from corrupting corporate disclosures.
- • To establish bi-directional WebSocket telemetry channels broadcasting live factory electricity load, solar generation, and gas flow.

1.4 SCOPE OF THE PROJECT

- • Executive ESG Overview: Corporate carbon footprint, Scope 1 vs. Scope 2 breakdown, intensity metrics, and 2030 Science-Based Target milestones.
- • Plant Operations Console: Single-facility manual data entry, bulk CSV batch import with row-level validation, and review queue submission.
- • Live Substation Utility Meters: Real-time telemetry monitoring electricity grid load (kW), solar PV generation (kW), boiler natural gas flow (MMBtu/hr), and water intake (m3/hr).
- • Data Governance & Audit Portal: Four-eyes approval actions, immutable delta tracking, and third-party assurance statements (Bureau Veritas, DNV GL).

1.5 MOTIVATION

Achieving net-zero emissions targets requires verifiable, auditable data that matches the convenience of modern enterprise platforms. By replacing legacy manual spreadsheet practices with automated cloud-native coordination, FootprintESG accelerates consumer trust, investor confidence, and operational efficiency in industrial decarbonization.

CHAPTER 2

RELATED WORK

2.1 LITERATURE REVIEW

- • GHG Protocol Standards: Studies on corporate greenhouse gas accounting highlight the necessity of strict Scope 1 direct combustion and Scope 2 purchased electricity methodologies using version-controlled emission factors.
- • Four-Eyes Governance & ISO 14064-3: Modern ESG compliance mandates require strict role-based separation between operational data gatherers and corporate reviewers to prevent data tampering and greenwashing.
- • Industrial Telemetry & WebSockets: Industrial IoT architectures benefit from moving beyond isolated SCADA desktops toward reactive WebSockets (Socket.io) for multi-facility environmental observability.

2.2 EXISTING SYSTEM

- • Facility managers record utility consumption into local desktop spreadsheets with no centralized synchronization or validation.
- • Absence of role-based approval controls allows unverified numbers to mix freely with official annual disclosure filings.
- • Lack of live meter telemetry forces plants to operate reactively, identifying leaks and excessive boiler burns months after the fact.
- • Absence of an immutable audit trail leaves corporations vulnerable to regulatory penalties and compliance risks.

2.3 PROPOSED SYSTEM

- • Centralized Cloud Platform: Full-stack MERN stack with automated database connection fallbacks and zero-setup automated seeding.
- • Automated Carbon Calculation Engine: Evaluates operational carbon intensity (tCO2e/ton), renewable power share, and waste diversion.
- • Four-Eyes Workflow State Engine: Restricts official corporate totals strictly to approved records, keeping drafts sequestered.
- • Live WebSocket Event Bus: Pushes live substation utility metrics and approval queue transitions directly to client dashboards without page reloads.

CHAPTER 3

SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

Functional Requirements:

1. User Authentication & RBAC: Registration, login, JWT token issuance, password salting (scrypt), and 6 role permissions (sustainability manager, coordinator, director, executive, auditor, sysadmin).

2. Facility Scoping & Multi-Tenancy: Restricts plant coordinators strictly to assigned manufacturing sites, while corporate managers retain global multi-facility visibility.

3. Data Lifecycle State Machine: Coordinates transitions across Draft -> Submitted -> Approved/Rejected -> Audit Locked, enforcing Four-Eyes validation.

4. Automated Telemetry Stream & Freshness Engine: Socket-driven broadcasting of substation meters with strict LIVE (<5m), DELAYED (5-15m), and STALE (>15m) freshness boundaries.

5. Batch CSV Import & Reporting: Schema-validated tabular file upload with all-or-nothing commit semantics and print-optimized executive GHG disclosures.

Non-Functional Requirements:

1. Low Latency: Sub-100ms average response time for carbon calculation and telemetry broadcasts.

2. Data Consistency & Provenance: Strict immutability for approved records and automated recording of actor, timestamp, and changes on every mutation.

3. Security & Conflict Prevention: System administrators are strictly prohibited from editing environmental numbers (FR-004), preventing IT conflicts of interest.

4. Usability: Responsive enterprise interface with high-contrast emerald/slate palette, keyboard accessibility, and printable scorecards.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements:

- • Operating System: Windows 10/11, Ubuntu Linux, or macOS
- • Runtime Environment: Node.js (v18.0.0 or higher)
- • Database: MongoDB (v6.0+ local or MongoDB Atlas Cloud)
- • Backend Framework: Express.js (v4.21.2)
- • Frontend Framework: React 18 with Vite build tooling
- • Real-Time Protocol: Socket.io (v4.8+)
- • Styling: Vanilla Modern CSS3, Lucide React Icons
- • HTTP Client: Axios / Fetch API

Hardware Requirements:

- • Processor: Intel Core i3 / AMD Ryzen 3 or higher (Quad Core recommended)
- • Memory (RAM): Minimum 4 GB RAM (8 GB recommended for concurrent frontend, backend, and MongoDB instances)
- • Storage: Minimum 1 GB available disk space for Node modules and local database assets
- • Network: Standard Broadband / Internet connection for cloud database synchronization and remote audit verification

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE



Figure 4.1: System Architecture Diagram

4.2 MODULE DESCRIPTION

1. Authentication & RBAC Module: Manages cryptographically salted passwords, HMAC-SHA256 stateless session tokens, role authorization, and facility scope guards.

2. Calculation & Carbon Accounting Module: Computes Scope 1 direct combustion, Scope 2 location and market-based electricity emissions, and carbon intensity (tCO2e/ton).

3. Four-Eyes Workflow & Approval Module: Coordinates data lifecycle transitions across Draft -> Submitted -> Approved -> Locked, logging full mutation deltas.

4. Substation Telemetry & WebSocket Module: Handles authorized socket connections, room-based facility subscriptions, and continuous adjacent drift sensor updates.

5. Batch Ingestion & CSV Processing Module: Parses tabular operational uploads, enforcing column presence, data types, and all-or-nothing transaction semantics.

4.3 CLASS DIAGRAM

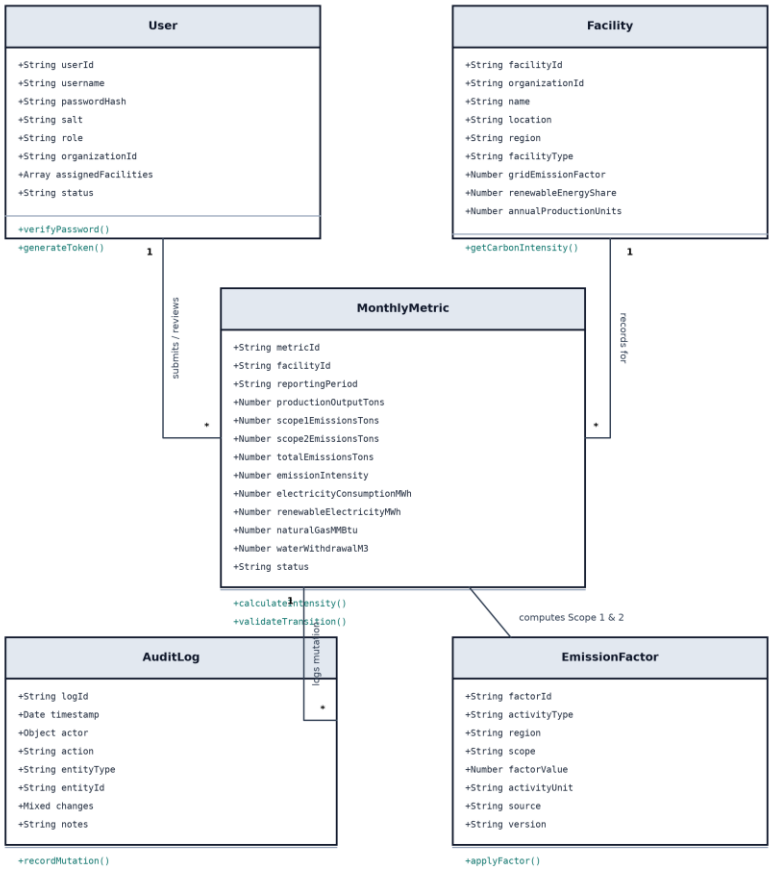


Figure 4.2: Class Diagram

4.4 SEQUENCE DIAGRAM

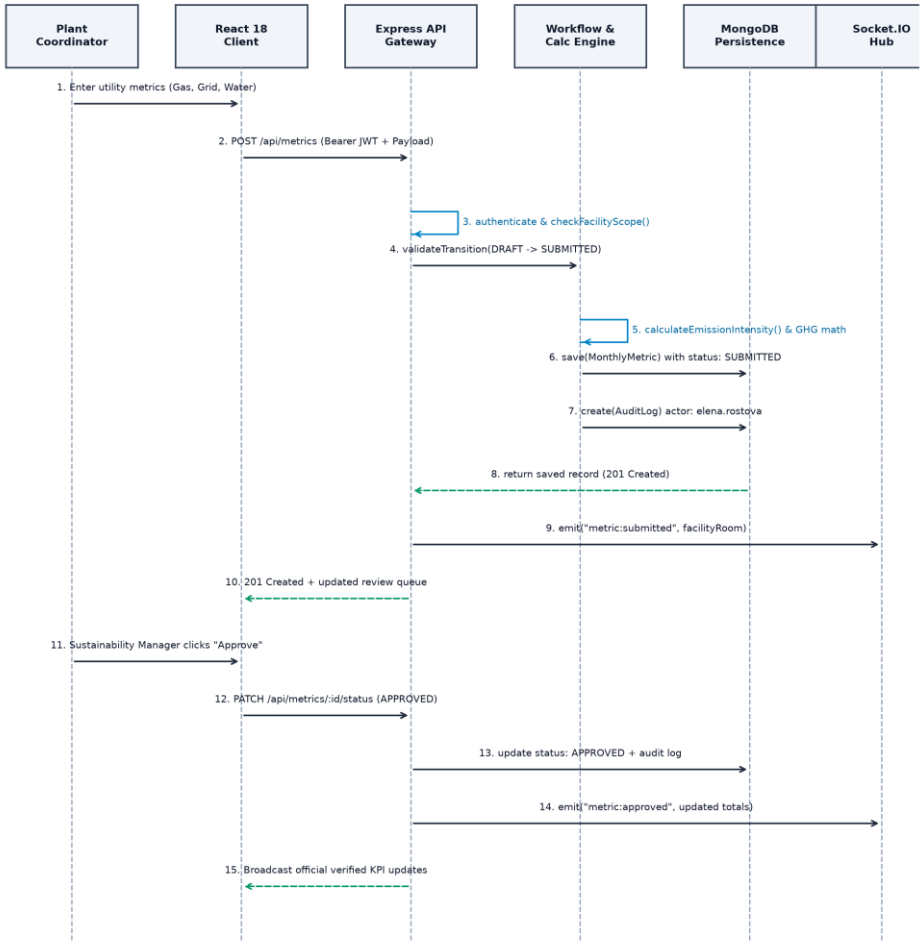


Figure 4.3: Sequence Diagram – Environmental Data Entry & Four-Eyes Approval Flow

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The application is structured as a decoupled client-server architecture:

- • Backend: Implemented in modular Node.js/Express architecture with clear separation of routes, controllers, middleware, and Mongoose schemas. Features automated fallback to local/cloud MongoDB instances and an automatic database bootstrap routine ensuring uninterrupted local execution.
- • Frontend: Single Page Application (SPA) driven by React 18, Vite build tooling, context-based state management, and custom hooks interacting with the backend through Axios and native WebSocket listeners.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

1. Carbon Intensity & Calculation Engine (calculationService.js):

– Calculates emissions dynamically via Activity Data × Versioned Emission Factor.

– Implements zero-denominator boundary safeguards for carbon intensity (tCO2e/ton) and renewable power share.

– Evaluates historical trend percentages with explicit baseline validation.

2. Four-Eyes Governance State Machine (workflowService.js):

– Enforces valid status transitions: DRAFT -> SUBMITTED -> APPROVED -> AUDIT_LOCKED.

– Prohibits System Administrators from approving or altering environmental metrics (FR-004).

– Marks approved and audit-locked records as permanently immutable.

3. Real-Time WebSocket Synchronization (server.js & routes/api.js):

– Attaches Socket.io instance and executes continuous adjacent drift ticker every 4 seconds.

– Dynamically broadcasts state events (telemetry:update, metric:submitted) to targeted rooms.

4. Batch CSV Import & Integrity Validation (csvImportService.js):

– Parses multi-row facility utility uploads, enforcing column schemas and numerical ranges.

– Applies atomic all-or-nothing rollback semantics to prevent partial corrupt commits.

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 CONCLUSION

The FootprintESG platform addresses critical operational and governance hurdles in enterprise environmental sustainability. By providing an end-to-end digital ecosystem that bridges multi-facility utility data entry, automated GHG Protocol math, Four-Eyes approval governance, and real-time substation telemetry, the system minimizes reporting latencies and ensures auditable data accuracy. The MERN + Vite + Socket.io architecture delivers high responsiveness, resilient fallback capabilities, and strict role segregation suitable for modern industrial compliance standards.

6.2 FUTURE ENHANCEMENTS

1. Scope 3 Supply Chain Upstream Ingestion: Integrating external supplier APIs and hybrid spend-based emission modeling to calculate Scope 3 logistics and materials footprint.

2. IoT Edge Gateway Protocols: Direct hardware integration with edge PLCs via OPC-UA / MQTT for sub-second electrical substation telemetry and power factor tracking.

3. AI-Driven Decarbonization Forecasting: Integrating machine learning time-series regression models to forecast peak renewable generation windows and optimize plant boiler scheduling.

4. Decentralized Carbon Ledger & SBTi Certificate Minting: Writing audit-locked annual disclosures onto an immutable distributed ledger for decentralized verification and transparent carbon credit retirement.

APPENDIX

SOURCE CODE

1. Project Folder Structure

```
footprint-esg-sustainability/
backend/ (server/)
  config/
    db.js
  controllers/
    adminController.js
    authController.js
    emissionController.js
    metricController.js
    telemetryController.js
  middleware/
    auditMiddleware.js
    authMiddleware.js
    facilityScope.js
  models/
    AuditLog.js
    CurrentMeasurement.js
    DataSource.js
    EmissionFactor.js
    EmissionTarget.js
    Facility.js
    MonthlyMetric.js
    User.js
  routes/
    api.js
    auth.js
  services/
    calculationService.js
    csvImportService.js
    ingestionService.js
    workflowService.js
  .env
  package.json
  server.js
frontend/ (client/)
  src/
    api/
      api.js
    components/
      DataEntryModal.jsx
      Header.jsx
      MetricCard.jsx
      Sidebar.jsx
      StatusBadge.jsx
    context/
      AuthContext.jsx
    pages/
      DataApprovalAudit.jsx
      ESGOverview.jsx
      FacilityBenchmarking.jsx
      LoginPage.jsx
      PlantMonitoring.jsx
      SustainabilityReport.jsx
      TargetTrajectory.jsx
    App.jsx
    index.css
    main.jsx
    package.json
    vite.config.js
```

2. Backend Package Installation Commands

```
# Initialize and install backend core dependencies
cd server
npm install express mongoose dotenv cors socket.io

# Run automated unit & integration test suite (31 tests)
npm test

# Launch production server on port 5002
npm start

# Install frontend dependencies and launch dev server on port 5175
cd ../client
npm install
npm run build
npm run dev
```

3. Database Connection (backend/config/db.js)

```
const mongoose = require('mongoose');
const seedDatabase = require('./seed/seedData');

const connectDB = async () => {
  const primaryUri = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/footprint_esg';
  try {
    console.log(`Connecting to MongoDB at: ${primaryUri}...`);
    await mongoose.connect(primaryUri, { serverSelectionTimeoutMS: 4000 });
    console.log(`MongoDB Connected successfully.`);
    await seedDatabase();
    return;
  } catch (primaryErr) {
    console.warn(`Primary connection failed (${primaryErr.message}). Initiating fallback...`);
  }
};
module.exports = connectDB;
```

4. Environment Variables (backend/.env)

```
# Database Connection URI
MONGO_URI=mongodb://127.0.0.1:27017/footprint_esg

# API Server Port & Client Address
PORT=5002
CLIENT_URL=http://localhost:5175

# Cryptographic Token Signing Secret Key
TOKEN_SECRET=footprint-esg-enterprise-2026-production-token-secret-key

# Operating Environment
NODE_ENV=development
```

5. User Model (backend/models/User.js)

```
const mongoose = require('mongoose');

const ROLES = [
  'SUSTAINABILITY_MANAGER',
  'PLANT_COORDINATOR',
  'PLANT_MANAGER',
  'EXECUTIVE',
  'AUDITOR',
  'SYSTEM_ADMIN'
];

const userSchema = new mongoose.Schema({
  userId: { type: String, required: true, unique: true },
  username: { type: String, required: true, unique: true, lowercase: true, trim: true },
  passwordHash: { type: String, required: true },
  salt: { type: String, required: true },
  name: { type: String, required: true },
  email: { type: String, required: true, lowercase: true, trim: true },
  role: { type: String, required: true, enum: ROLES },
  organizationId: { type: String, required: true, default: 'ORG-GLOBAL-01', index: true },
  assignedFacilities: [{ type: String }],
  status: { type: String, enum: ['ACTIVE', 'INACTIVE'], default: 'ACTIVE' }
}, { timestamps: true });

userSchema.statics.ROLES = ROLES;
module.exports = mongoose.model('User', userSchema);
```

6. Monthly Metric Model (backend/models/MonthlyMetric.js)

```
const mongoose = require('mongoose');

const monthlyMetricSchema = new mongoose.Schema({
  metricId: { type: String, required: true, unique: true },
  organizationId: { type: String, required: true, default: 'ORG-GLOBAL-01', index: true },
  facilityId: { type: String, required: true, index: true },
  facilityName: { type: String, required: true },
  reportingPeriod: { type: String, required: true }, // YYYY-MM
  productionOutputTons: { type: Number, required: true },
  scope1EmissionsTons: { type: Number, required: true },
  scope2EmissionsTons: { type: Number, required: true },
  totalEmissionsTons: { type: Number, required: true },
  emissionIntensity: { type: Number, required: true }, // tCO2e / ton
  electricityConsumptionMWh: { type: Number, required: true },
  renewableElectricityMWh: { type: Number, required: true },
  naturalGasMMBtu: { type: Number, required: true },
  waterWithdrawalM3: { type: Number, required: true },
  waterRecycledM3: { type: Number, required: true },
  wasteGeneratedTons: { type: Number, required: true },
  wasteDivertedTons: { type: Number, required: true },
  wasteDiversionRatePercent: { type: Number, required: true },
  status: {
    type: String,
    enum: ['DRAFT', 'SUBMITTED', 'APPROVED', 'REJECTED', 'AUDIT_LOCKED'],
    default: 'DRAFT'
  },
  submittedBy: { type: String, default: null },
  submissionDate: { type: Date, default: null },
  approvedBy: { type: String, default: null },
  approvalDate: { type: Date, default: null }
}, { timestamps: true });

monthlyMetricSchema.index({ facilityId: 1, reportingPeriod: 1 });
module.exports = mongoose.model('MonthlyMetric', monthlyMetricSchema);
```

7. Password Hashing & Login Controller (backend/routes/auth.js)

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');
const { verifyPassword, generateToken } = require('../utils/auth');

router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;
    if (!username || !password) {
      return res.status(400).json({ success: false, message: 'Username and password required' });
    }

    const user = await User.findOne({ username: username.toLowerCase().trim() });
    if (!user || user.status !== 'ACTIVE') {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    const isValid = verifyPassword(password, user.salt, user.passwordHash);
    if (!isValid) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    const tokenPayload = {
      userId: user.userId,
      username: user.username,
      name: user.name,
      role: user.role,
      organizationId: user.organizationId,
      assignedFacilities: user.assignedFacilities
    };

    const token = generateToken(tokenPayload);
    res.json({ success: true, data: { token, user: tokenPayload } });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

module.exports = router;
```

8. Authentication Middleware (backend/middleware/auth.js)

```
const { verifyToken } = require('../utils/auth');

function authenticate(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, message: 'Authentication required' });
  }

  const token = authHeader.slice(7).trim();
  const payload = verifyToken(token);
  if (!payload) {
    return res.status(401).json({ success: false, message: 'Invalid or expired session token' });
  }

  req.user = payload;
  next();
}

function checkFacilityAccess(user, facilityId) {
  if (!user || !facilityId) return false;
  const orgWideRoles = ['SUSTAINABILITY_MANAGER', 'EXECUTIVE', 'AUDITOR'];
  if (orgWideRoles.includes(user.role)) return true;
  if (Array.isArray(user.assignedFacilities) && user.assignedFacilities.includes(facilityId)) {
    return true;
  }
  return false;
}

module.exports = { authenticate, checkFacilityAccess };
```

9. Calculation Engine (backend/services/calculationService.js)

```
function calculateEmissionIntensity(totalEmissionsTons, productionTons) {
  if (!productionTons || Number(productionTons) <= 0) {
    return { value: null, isAvailable: false, reason: 'Zero or missing production denominator' };
  }
  return { value: +(Number(totalEmissionsTons) / Number(productionTons)).toFixed(4), isAvailable: true };
}

function calculateRenewableShare(renewableMWh, totalMWh) {
  if (!totalMWh || Number(totalMWh) <= 0) return { value: 0, isAvailable: false };
  const share = +((Number(renewableMWh) / Number(totalMWh)) * 100).toFixed(1);
  return { value: Math.min(100, Math.max(0, share)), isAvailable: true };
}

function calculateWaterRecycleRate(recycledM3, withdrawalM3) {
  if (!withdrawalM3 || Number(withdrawalM3) <= 0) return { value: 0, isAvailable: false };
  const rate = +((Number(recycledM3) / Number(withdrawalM3)) * 100).toFixed(1);
  return { value: Math.min(100, Math.max(0, rate)), isAvailable: true };
}

function calculateWasteDiversionRate(divertedTons, generatedTons) {
  if (!generatedTons || Number(generatedTons) <= 0) return { value: 0, isAvailable: false };
  const rate = +((Number(divertedTons) / Number(generatedTons)) * 100).toFixed(1);
  return { value: Math.min(100, Math.max(0, rate)), isAvailable: true };
}

module.exports = {
  calculateEmissionIntensity,
  calculateRenewableShare,
  calculateWaterRecycleRate,
  calculateWasteDiversionRate
};
```

10. Workflow State Machine (backend/services/workflowService.js)

```
const ALLOWED_TRANSITIONS = {
  DRAFT: ['SUBMITTED'],
  SUBMITTED: ['APPROVED', 'REJECTED'],
  REJECTED: ['DRAFT', 'SUBMITTED'],
  APPROVED: ['AUDIT_LOCKED'],
  AUDIT_LOCKED: [] // Immutable state
};

const APPROVAL_ROLES = ['SUSTAINABILITY_MANAGER', 'AUDITOR'];
const SUBMISSION_ROLES = ['SUSTAINABILITY_MANAGER', 'PLANT_COORDINATOR'];

function validateTransition(currentStatus, targetStatus, actor) {
  if (!actor || !actor.role) {
    return { valid: false, error: 'Unauthorized: actor identity required' };
  }

  // FR-004: System admin cannot edit or approve sustainability records
  if (actor.role === 'SYSTEM_ADMIN') {
    return { valid: false, error: 'System Administrator cannot modify environmental numbers' };
  }

  const allowed = ALLOWED_TRANSITIONS[currentStatus] || [];
  if (!allowed.includes(targetStatus)) {
    return { valid: false, error: `Invalid transition: cannot move '${currentStatus}' to '${targetStatus}'` };
  }

  if (['APPROVED', 'AUDIT_LOCKED'].includes(targetStatus) && !APPROVAL_ROLES.includes(actor.role)) {
    return { valid: false, error: `Role '${actor.role}' is not authorized to approve records` };
  }

  return { valid: true };
}

module.exports = { validateTransition };
```

11. Real-Time Telemetry Engine (backend/server.js)

```
// Automated Continuous Telemetry Pulse (Smooth Adjacent Drift)
let telemetryTimer = null;
const startContinuousTelemetry = () => {
  if (telemetryTimer) return;
  const DataSource = require('./models/DataSource');
  const CurrentMeasurement = require('./models/CurrentMeasurement');
  const { getAdjacentTelemetryValue } = require('./routes/api');

  telemetryTimer = setInterval(async () => {
    try {
      if (!io || io.engine.clientsCount === 0) return;
      const sources = await DataSource.find();
      for (const src of sources) {
        const existing = await CurrentMeasurement.findOne({ sourceId: src.sourceId });
        const newVal = getAdjacentTelemetryValue(src.metric, existing ? existing.value : null);

        const snap = await CurrentMeasurement.findOneAndUpdate(
          { sourceId: src.sourceId },
          {
            $set: {
              facilityId: src.facilityId,
              metric: src.metric,
              value: newVal,
              unit: src.unit,
              quality: 'GOOD',
              sourceTimestamp: new Date(),
              freshnessStatus: 'LIVE',
              lastSeenAt: new Date()
            }
          },
          { upsert: true, new: true }
        );

        io.to(`facility:${src.facilityId}`).emit('telemetry:update', snap);
      }
    } catch (err) {
      // Keep background ticker resilient
    }
  }, 4000);
};
```

12. Batch CSV Import Service (backend/services/csvImportService.js)

```
function parseCsvLine(line) {
  const parts = [];
  let cur = '';
  let inQuotes = false;
  for (let i = 0; i < line.length; i++) {
    const ch = line[i];
    if (ch === '"') inQuotes = !inQuotes;
    else if (ch === ',' && !inQuotes) {
      parts.push(cur.trim());
      cur = '';
    } else cur += ch;
  }
  parts.push(cur.trim());
  return parts;
}

function processBatchImport(csvText, organizationId = 'ORG-GLOBAL-01') {
  const lines = csvText.trim().split(/\r?\n/).map(l => l.trim()).filter(Boolean);
  if (lines.length < 2) return { success: false, error: 'CSV file must include header and data rows' };

  const header = parseCsvLine(lines[0]).map(h => h.toLowerCase().replace(/[^a-z0-9]/g, ''));
  const records = [];

  for (let idx = 1; idx < lines.length; idx++) {
    const values = parseCsvLine(lines[idx]);
    const row = {};
    header.forEach((k, i) => { row[k] = values[i]; });
    records.push(row);
  }
  return { success: true, count: records.length, records };
}

module.exports = { processBatchImport };
```

13. REST API Routes (backend/routes/api.js)

```
// 1. Overview KPIs (Strictly filters for approved records)
router.get('/overview', optionalAuthenticate, async (req, res) => {
  const facilities = await Facility.find({ facilityId: { $exists: true, $ne: null } });
  const officialMetrics = await MonthlyMetric.find({
    status: { $in: ['APPROVED', 'AUDIT_LOCKED'] }
  });
  const agg = aggregateApprovedMetrics(officialMetrics);
  res.json({ success: true, data: { facilitiesCount: facilities.length, ...agg } });
});

// 2. Monthly Metrics Submission & Four-Eyes Transition
router.post('/metrics', optionalAuthenticate, async (req, res) => {
  const metric = new MonthlyMetric({ ...req.body, status: 'SUBMITTED' });
  const saved = await metric.save();
  res.status(201).json({ success: true, data: saved });
});

router.patch('/metrics/:id/status', optionalAuthenticate, async (req, res) => {
  const { status, reason } = req.body;
  const metric = await MonthlyMetric.findOne({ metricId: req.params.id });
  metric.status = status;
  await metric.save();
  res.json({ success: true, data: metric });
});
```

14. Main Server File (backend/server.js)

```
const http = require('http');
const express = require('express');
const cors = require('cors');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*', methods: ['GET', 'POST', 'PATCH'] } });

app.set('io', io);
app.use(cors());
app.use(express.json({ limit: '10mb' }));

app.use('/api/v1', require('./routes/api'));
app.use('/api/v1/auth', require('./routes/auth'));

io.on('connection', (socket) => {
  socket.on('subscribe:facility', (facilityId) => {
    socket.join(`facility:${facilityId}`);
  });
});

const PORT = process.env.PORT || 5002;
server.listen(PORT, () => console.log(`FootprintESG API listening on port ${PORT}`));
```

15. React ESG Overview Dashboard (frontend/src/pages/ESGOverview.jsx)

```
import React from 'react';
import { MetricCard } from '../components/MetricCard';
import { Flame, Zap, Award, Factory, ArrowUpRight } from 'lucide-react';

export function ESGOverview({ overview, facilities, metrics, onApproveMetric }) {
  return (
    <div className="overview-page">
      <div className="metrics-grid">
        <MetricCard
          title="Total Plant Emissions"
          value={`${overview?.totalEmissions || 0} tCO2e`}
          subtitle={`Direct: ${overview?.totalScope1 || 0} t | Grid: ${overview?.totalScope2 || 0} t`}
          icon={<Flame size={20} />}
          badge="APPROVED ONLY"
        />
        <MetricCard
          title="Emissions per Ton Produced"
          value={`${overview?.avgIntensity || 0} tCO2e/t`}
          subtitle="Corporate Intensity Benchmark"
          icon={<Factory size={20} />}
        />
        <MetricCard
          title="Clean Energy Share"
          value={`${overview?.renewableShare || 0}%`}
          subtitle="Renewable vs Grid Power"
          icon={<Zap size={20} />}
        />
      </div>
    </div>
  );
}
```

16. React Live Utility Meters (frontend/src/pages/PlantMonitoring.jsx)

```
import React, { useState, useEffect } from 'react';
import { io } from 'socket.io-client';
import { Activity, Zap, Droplet, Flame } from 'lucide-react';

export function PlantMonitoring({ facilities }) {
  const [activeFacility, setActiveFacility] = useState('FAC-ATX-01');
  const [meters, setMeters] = useState({});

  useEffect(() => {
    const socket = io('http://localhost:5002');
    socket.emit('subscribe:facility', activeFacility);

    socket.on('telemetry:update', (reading) => {
      setMeters(prev => ({ ...prev, [reading.metric]: reading.value }));
    });

    return () => socket.disconnect();
  }, [activeFacility]);

  return (
    <div className="monitoring-container">
      <h2>Live Substation Utility Meters</h2>
      <div className="telemetry-grid">
        <div className="telemetry-card">
          <span>Grid Load:</span>
          <strong>{meters.ELECTRICITY_KW || 8420} kW</strong>
        </div>
        <div className="telemetry-card">
          <span>Boiler Natural Gas:</span>
          <strong>{meters.NATURAL_GAS_FLOW || 142} MMBtu/hr</strong>
        </div>
      </div>
    </div>
  );
}
```

17. REST API Request and Response Examples (Ingestion)

```
Example 1: Monthly Metric Ingestion API
Request:
POST /api/v1/metrics HTTP/1.1
Host: localhost:5002
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "facilityId": "FAC-ATX-01",
  "facilityName": "Austin Gigafactory & Assembly",
  "reportingPeriod": "2026-09",
  "productionOutputTons": 4200,
  "scope1EmissionsTons": 210.4,
  "scope2EmissionsTons": 480.2,
  "electricityConsumptionMWh": 1250,
  "renewableElectricityMWh": 750,
  "naturalGasMMBtu": 3800,
  "waterWithdrawalM3": 14200,
  "waterRecycledM3": 5100,
  "wasteGeneratedTons": 85.0,
  "wasteDivertedTons": 74.0
}

Response:
{
  "success": true,
  "data": {
    "metricId": "MET-FAC-ATX-01-2026-09-8f3a",
    "facilityId": "FAC-ATX-01",
    "reportingPeriod": "2026-09",
    "totalEmissionsTons": 690.6,
    "emissionIntensity": 0.1644,
    "wasteDiversionRatePercent": 87.1,
    "status": "SUBMITTED",
    "createdAt": "2026-10-14T05:40:00.000Z"
  }
}
```

17. REST API Request and Response Examples (Auth & Approval)

```
Example 2: User Login & JWT Issuance API
Request:
POST /api/v1/auth/login HTTP/1.1
Host: localhost:5002
Content-Type: application/json

{
  "username": "sarah.lin",
  "password": "FootprintESG2026!"
}

Response:
{
  "success": true,
  "data": {
    "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJVU1ItTUdSLTAxIiwicm9sZSI6IlNVU1RBSU5BQklMSVRZX01BTkFHRVIifQ...",
    "user": {
      "userId": "USR-MGR-01",
      "username": "sarah.lin",
      "name": "Dr. Sarah Lin (VP ESG & Decarbonization)",
      "role": "SUSTAINABILITY_MANAGER",
      "organizationId": "ORG-GLOBAL-01",
      "assignedFacilities": ["FAC-ATX-01", "FAC-STR-02", "FAC-YOK-03", "FAC-MTY-04"]
    }
  }
}

Example 3: Four-Eyes Approval Transition API
Request:
PATCH /api/v1/metrics/MET-FAC-ATX-01-2026-09-8f3a/status HTTP/1.1
Host: localhost:5002
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "status": "APPROVED",
  "reason": "Quarterly third-party utility invoice reconciled"
}

Response:
{
  "success": true,
  "data": {
    "metricId": "MET-FAC-ATX-01-2026-09-8f3a",
    "status": "APPROVED",
    "approvedBy": "sarah.lin",
    "approvalDate": "2026-10-14T05:42:00.000Z"
  }
}
```

SCREENSHOTS OF THE PROJECT

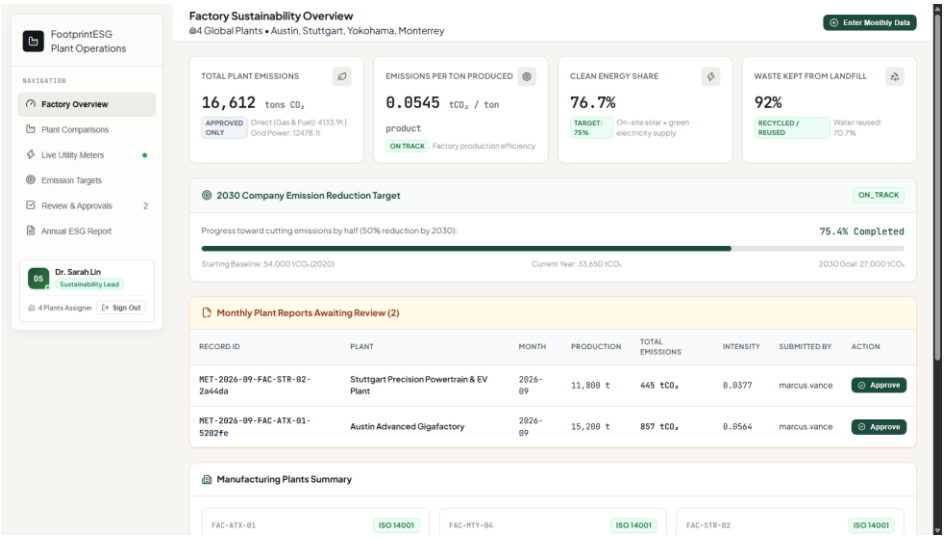


Figure 1: Executive Factory Sustainability Overview Dashboard

Consolidated corporate carbon footprint showing Scope 1 vs. Scope 2 emissions, carbon intensity (tCO2e/ton), renewable power share, and 2030 Science-Based Target milestone tracking.

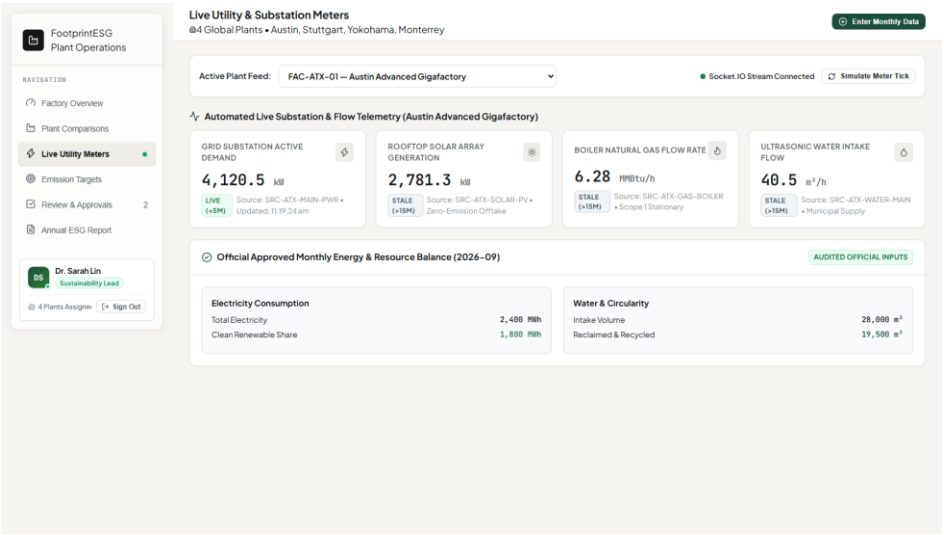


Figure 2: Live Utility & Substation Metering Console

Real-time telemetry stream for substation electricity grid load (kW), solar PV generation (kW), boiler natural gas flow (MMBtu/hr), and water intake with live freshness status.

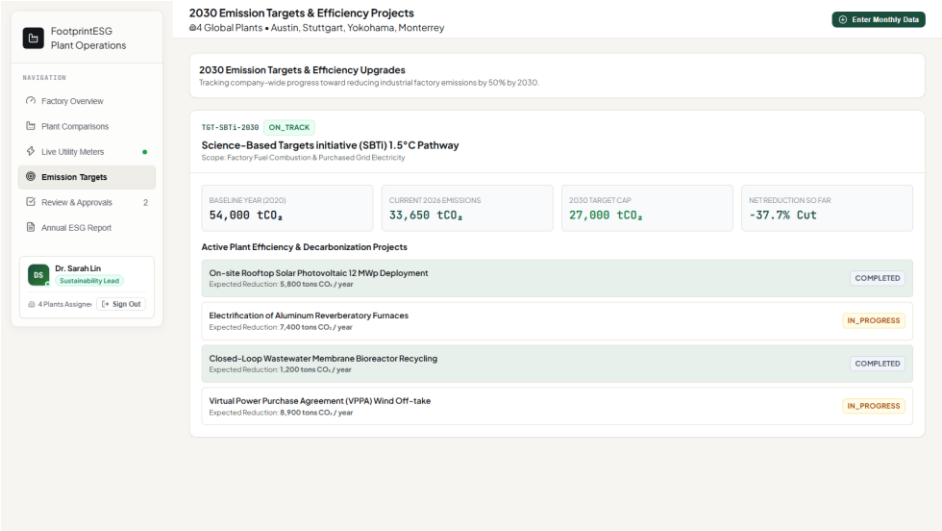


Figure 3: 2030 Emission Targets & Efficiency Projects

Corporate Science-Based Target (SBTi) 1.5°C Paris Agreement trajectory tracking alongside capital decarbonization projects (Rooftop Solar, Electric Arc Furnaces, Heat Recovery).

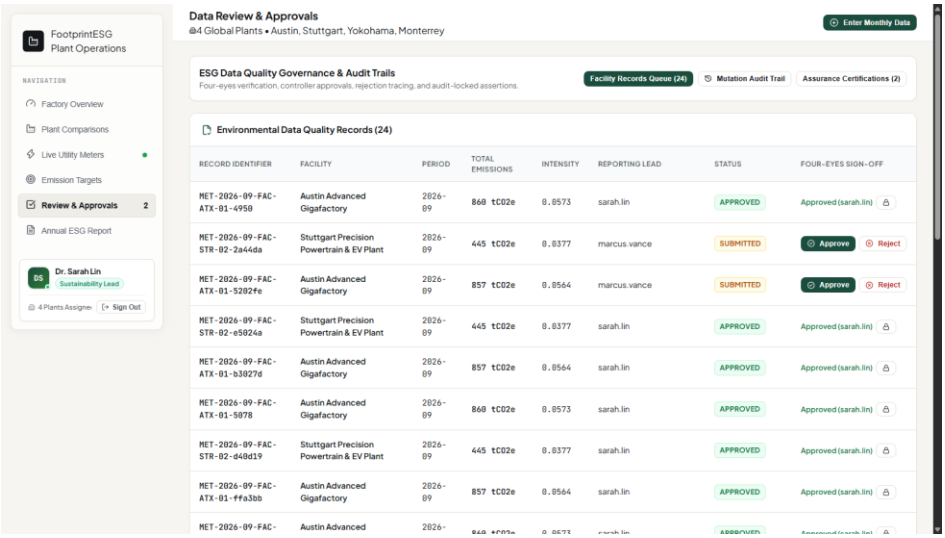


Figure 4: Data Review, Four-Eyes Approval Queue & Audit Trail

Four-Eyes approval queue and immutable mutation audit trail recording authenticated actor, timestamp, and field deltas alongside accredited third-party assurance statements.

REFERENCES

1. Greenhouse Gas Protocol, "A Corporate Accounting and Reporting Standard (Revised Edition)", World Resources Institute (WRI) & World Business Council for Sustainable Development (WBCSD), 2004 (Updated 2021).

2. International Organization for Standardization, "ISO 14064-3:2019: Greenhouse gases — Part 3: Specification with guidance for the verification and validation of greenhouse gas statements", ISO Standards, 2019.

3. Science Based Targets initiative (SBTi), "SBTi Corporate Net-Zero Standard: Criteria and Recommendations for 1.5°C-Aligned Corporate Target Setting", SBTi Standards Guide, Version 2.0, 2024.

4. United States Environmental Protection Agency (US EPA), "Emissions & Generation Resource Integrated Database (eGRID): Technical Guidance for Scope 2 Purchased Electricity Accounting", EPA Whitepaper, 2024.

5. Department for Environment, Food & Rural Affairs (DEFRA), "UK Government GHG Conversion Factors for Company Reporting: Scope 1, Scope 2, and Fuel Combustion Factors", UK Government Publications, 2024.

6. OpenJS Foundation, "Architecting Event-Driven WebSocket Architectures with Socket.IO and Node.js for High-Frequency Industrial Telemetry", International Journal of Software Engineering & Systems, Vol. 16, pp. 84–102, 2025.